```
================================================================================
TIME-TRAVEL CLI v1.6 - FILE REPOS + EXTLOG + AUTOTAGS + METADATA ENCRYPTION
Pure C11 engine. Static binary. Linux only.

"TimeTravel is for the battle; Git is for after the battle."
================================================================================
"Serious versioning for those who want Ctrl+Z across their entire project."

Language:    C11 (engine and CLI), Zero runtime dependencies.
Binary:      Static CLI (musl-gcc + integrated xdelta3, BLAKE2b and crypto).
Platforms:   x86_64, ARM64, MIPS64 (MSB), RISC-V
Format:      TTDLTA01, packed little-endian records with xdelta3 deltas.
Crypto:      Optional XChaCha20-Poly1305 payload encryption.
Dedup:       BLAKE2b content-addressed 64 KiB dedup blocks.
Status:      Production. Core validated with 170 assertions across 52 sections.

Functional battery: 170 assertions.
Dedicated Valgrind memory audit: 8 assertions.
TSan mode evaluates the functional assertions and skips the Valgrind memory audit
automatically.

v1.4:
DEDUP + ENCRYPTION.
Files >= 1 MiB can be captured as 64 KiB BLAKE2b
content-addressed blocks. Repos can be encrypted with
XChaCha20-Poly1305. Adds store verification, atomic
compaction, orphan-block GC and hardened failure paths.

v1.5:
AUTOTAGS + METADATA ENCRYPTION.
Semantic auto-tags in log output (+1/-1 lines 3 #max_retries).
Transparent metadata encryption:
annotations.enc, baseline.enc, exclude.enc are AEAD-encrypted with a per-repo key. Modular
architecture refactor: tt_capture, tt_repo,tt_cmds, tt_reconstruct, tt_exclude, tt_autotag,
tt_annotation. Unified reconstruction engine.

v1.6:
FILE REPOS + EXTLOG + OPTIMIZED TESTS.
Support for watching individual files (not just directories).
New 'extlog' command for detailed semantic history.
Optimized compilation modes (test/release) for faster regression testing.

================================================================================
DEVELOPMENT AND TESTING ENVIRONMENT
================================================================================
All code and the 170 assertions / 51-section regression battery were developed and
validated on the
following hardware, without access to modern machines during the process:

CPU:       Intel Core 2 Duo T7500 @ 2.20 GHz (2 cores, year 2007)
OS:        Linux Mint 21 x86_64 (kernel 5.15 LTS)
RAM:       4 GB
Disk:      Mechanical HDD (no SSD)

Toolchain: musl-gcc, gcc sanitizer builds, xdelta3, Valgrind

Key results on this hardware:
- 170 assertions passing when the dedicated Valgrind memory audit is included.
- TSan mode: 116+ evaluated assertions + 1 skipped Valgrind section.
- AddressSanitizer / UndefinedBehaviorSanitizer audit builds available.
- ThreadSanitizer audit build available.
```

- Valgrind memory audit: 0 leaks, 0 errors.
- Atomic restoration via temp+rename: zero partial writes.
- Store verification detects corrupted or tampered .ttd files, including invalid magic.
- Dedup restore validated with 2 MiB random files.
- Dedup GC removes orphan blocks while preserving referenced blocks.
- Encrypted stores leave no plaintext in .ttd files.
- Encrypted compact leaves no plaintext behind.
- Wrong passphrase does not modify or garbage-collect dedup blocks.
- Corrupted dedup blocks are detected during restore.
- Corrupted crypto.meta is rejected.

If it works here, it works anywhere.

NOTE: All times mentioned were measured in the environment described above.
On modern hardware (NVMe SSD, multi-core CPU), results are orders of
magnitude better.

==
QUICK START
==
Compile:
./compile.sh test   # Fast KDF for tests (~100x faster crypto tests)
./compile.sh release   # Full security for production (default if no arg provided in older versions, now required)

Watch a single directory (starts the monolith):
./build/timetravel start /path/to/project

or watch a single file (v1.6):
./build/timetravel start /path/to/project/file.c

or multiple targets at start:
./build/timetravel start src/ bin/ main.c

Add MORE repositories/files hot to the same daemon:
./build/timetravel add /path/to/another/project
./build/timetravel add /path/to/docs/readme.md

See the whole swarm from ANY directory:
./build/timetravel status

Edit files normally. To undo:
./build/timetravel undo /path/to/file.c --last --repo /path/to/project
./build/timetravel undo /path/to/file.c --to "2026-09-13 00:07:08" --repo /path/to/project
./build/timetravel undo /path/to/file.c --to "10 minutes ago" --repo /path/to/project
./build/timetravel undo /path/to/project --tag before-refactor --force --repo /path/to/project

To compare versions:
./build/timetravel diff /path/to/file.c --repo /path/to/project
./build/timetravel diff /path/to/file.c --to prev --repo /path/to/project
./build/timetravel diff /path/to/file.c --to 15 --repo /path/to/project

To see inter-revision deltas, use dump --with-diff instead.
To view history and status:
./build/timetravel log /path/to/file.c --repo /path/to/project
./build/timetravel extlog /path/to/file.c --repo /path/to/project # New in v1.6
./build/timetravel status --repo /path/to/project
./build/timetravel tag release-v1.0 --repo /path/to/project

To export all versions and diffs:

```
./build/timetravel dump /path/to/file.c --repo /path/to/project \
--out /tmp/tt_dump --with-diff

Lifecycle and maintenance:
./build/timetravel stop                    # stop the whole monolith
./build/timetravel stop --repo <dir>       # remove ONLY that repo
./build/timetravel restart <dir>
./build/timetravel tags [--repo <dir>]
./build/timetravel compact [--repo <dir>]  # manual (no more 03:00 auto-run)

Create an encrypted repository:
./build/timetravel start /path/to/project --encrypt

This prompts for a passphrase twice and creates:
/path/to/project/.timetravel/crypto.meta

Verify repository integrity:
./build/timetravel verify --repo /path/to/project

Compact a repository and run dedup GC:
./build/timetravel compact --repo /path/to/project

For encrypted repositories, compact/undo/verify may request the passphrase.

Run regression tests:
./compile.sh test
./mega_test.sh ./build/timetravel

Run sanitizer and concurrency audits:
# AddressSanitizer + UndefinedBehaviorSanitizer
./compile_asan.sh
./mega_test.sh ./build_san/timetravel

# ThreadSanitizer
./compile_tsan.sh
TSAN_OPTIONS="halt_on_error=1:second_deadlock_stack=1" \
./mega_test.sh ./build_tsan/timetravel

Important:
- Do not combine Valgrind with ASan or TSan.
- Do not combine ASan with TSan.
- When mega_test detects a TSan build, it automatically skips the Valgrind
memory audit section.
- Sanitizer builds are audit builds, not the production static binary.

================================================================================
[TOC]
================================================================================
1.  What is Time-Travel
2.  Engine Architecture
    2.1 The multi-repo monolith
    2.2 Capture pipeline diagram
    2.3 Restoration pipeline diagram
    2.4 Export (dump) diagram
3.  On-Disk Store Structure
4.  Binary Format
5.  Delta Encoding with xdelta3
6.  Capture Model (Copy-on-First-Change)
7.  CLI - Command Reference
    7.1 Daemon lifecycle (multi-repo)
    7.2 Restoration
```

    7.3 Inspection and comparison
    7.4 Export
    7.5 Maintenance
    7.6 Time and revision expressions
    7.7 Confirmation behavior
8. Revision and Time Expressions
9. Version and Diff Export (dump)
10. Code Quality and Coverage (170 assertions / 52 sections)
    10.1 Certification summary
    10.2 Forensic memory and concurrency audits
    10.3 Valgrind / sanitizer policy (v1.4)
11. Design Decisions
12. Limits and Constants
13. Possible Issues
14. Dedup block storage (V1.4)
15. Encryption (V1.4)
    15.1 Semantic Autotags (V1.5)
    15.2 Metadata Encryption (V1.5)
16. FAQ - Frequently Asked Questions
    16.1 Can i use time-travel on windows or mac?
    16.2 Does it work over nfs / sshfs / network mounts?
    16.3 How much disk space does the store need?
    16.4 Can i backup the .timetravel directory?
    16.5 What happens if the daemon crashes?
    16.6 Can two daemons watch the same directory?
    16.7 Can i watch several unrelated folders at once? (new in v1.3)
    16.8 Is there encryption?
    16.9 Can i sync the store to another machine?
    16.10 How do i resume a session?
    16.11 Can i see 100 diffs?
    16.12 How do i see the change introduced by a specific revision?
    16.13 Can i use exact dates from the log?
17. Comparison with Other Tools
18. Resuming a Session
19. Anecdote
20. Changelog
21. Timetravel license

1. WHAT IS TIME-TRAVEL

Time-Travel is a real-time file versioning daemon that watches a directory tree or individual files via inotify and records every change as compact xdelta3 deltas.

Think of it as "Ctrl+Z for your entire project folder":

Core capture and restoration:
- Every edit, creation, and deletion is captured automatically.
- Restore any file or entire directory to any previous point in time.
- Named checkpoints (tags) for semantic rollback points.
- Diff between any two versions of a file.
- Full export of reconstructed versions and consecutive diffs.
- Atomic writes: restorations never leave corrupted files.
- No initial backup: only changes are recorded (copy-on-first-change).
- Per-file directory undo: each file reverts to its own previous version.

Multi-repo monolith (v1.3):
- ONE daemon watches up to 16 repositories at once.
- Hot add/remove of repositories without restarting.
- Global status across the whole swarm from any directory.

- Per-repo isolation: corruption in one repo never affects the others.

File Repos (v1.6):
- Watch individual files instead of entire directories.
- Perfect for config files, single scripts, or isolated modules.
- Shares the same daemon and store infrastructure as directory repos.

Dedup and encryption (v1.4):
- Files >= 1 MiB are stored as 64 KiB BLAKE2b content-addressed blocks, shared across files and versions.
- Optional per-repository XChaCha20-Poly1305 encryption (AEAD).
- PBKDF2-HMAC-BLAKE2b key derivation; the passphrase is never stored.
- Store verification: detect corrupted or tampered .ttd files, invalid magic, missing blocks, and failed AEAD authentication.
- Atomic compaction with fail-closed orphan-block GC.

Reliability and hygiene:
- Robust exclusions: .git, node_modules, __pycache__, target, .cache.
- Enhanced status showing daemon state, CPU, RAM, records, and bytes.
- Self-cleaning test battery: 170 assertions across 51 sections.
- Audited with ASan/UBSan, ThreadSanitizer and Valgrind (0 leaks).

The final CLI binary is static, approximately 274 KB (includes statically compiled xdelta3, BLAKE2b, and the XChaCha20-Poly1305 crypto core), with zero external runtime dependencies.

2. ENGINE ARCHITECTURE

The engine is organized in C11 modules with clear responsibilities.

FILE	RESPONSIBILITY
tt_types.h	Base types, constants, event types (CREATE, MODIFY, DELETE, RENAME, CREATE_DEDUP), record layout (packed 25-byte header), dedup/crypto constants, TtDaemon/TtLocalRepo with v1.6 file_filter field
tt_main.c	CLI entry point, daemon poll loop (inotify + timer + IPC), v1.6 multi-file start/add (up to 10 paths, glob expansion, batch quiet mode), monolith orchestration, signal handling
tt_store.c	.ttd read/write, rotation at 64 MiB, scan_stats, magic validation, encrypted payload I/O (nonce + ciphertext + tag), per-repo TtStore context, TtStoreReader with crypto key inheritance
tt_repo.c	Repo lifecycle: adoption (base dir + v1.6 single-file), PID/status files, root health recovery, repos.list persistence (supports path|file_filter format), overlap detection, selective release
tt_capture.c	Capture pipeline: CREATE/CREATE_DEDUP/MODIFY/DELETE record writing, capture_path with v1.6 file_filter isolation, initial build walk, periodic rescan, debounce callback, autotag generation on capture
tt_cmds.c	CLI command handlers: verify, watch, add, stop, undo (time/last/initial/tag modes), status (global + per-repo), log, v1.6 extlog (revisions, ratios,

	elapsed time, semantic summary), compact, tag, diff, dump (with CREATE_D support), exclude
tt_watcher.c	inotify management, recursive watch setup, wd?repo routing, v1.6 multi-repo event fan-out (several single-file repos sharing a parent dir), root health detection, queue overflow recovery
tt_ipc.c	Unix-socket IPC for hot add/remove/list, global registry (/tmp/timetravel-<uid>/), boot lock (anti-stampede), client/server dispatch
tt_restore.c	File/dir restoration, per-file undo, atomic writes via temp+rename, dedup block reassembly, v1.6 baseline.enc (encrypted) with legacy fallback, undo --initial prunes files added after baseline, encrypted restore
tt_reconstruct.c	Unified reconstruction engine: point-in-time state machine (CREATE ? MODIFY chain ? DELETE), replaces duplicated logic in restore/diff/dump/verify
tt_history.c	History queries: TtPathSet, tags (add/lookup/list), scan_timestamps, resolve_file_target_ns (supports "now", "prev", "first", "@ts", "N", "-N", "tag:name", relative expressions), store collection, scan_stats_dir
tt_dedup.c	64 KiB BLAKE2b-256 content-addressed block pool, block storage/retrieval, convergent encryption (key = BLAKE2b(repo_key || block_hash)), split/reconstruct, hardened fail-closed GC (HARDENED-GC-V3) with sorted hash vector
tt_compact.c	Atomic compaction: delta chain collapse (threshold 15), super-delta encoding, temp-dir write + rename, dedup GC invocation, encrypted compact support
tt_crypto.c	XChaCha20-Poly1305 AEAD (RFC 8439 + draft-xchacha), Poly1305 radix-2^26, HChaCha20, path encryption with AAD, crypto.meta (magic + salt + iters + verifier), v1.6 tt_repo_get_key (transparent 32-byte key), encrypt/decrypt full files (TTENC01 format)
tt_kdf.c	PBKDF2-HMAC-BLAKE2b passphrase derivation, RFC 2104 HMAC over BLAKE2b (128-byte block), configurable iterations (100000 prod / 1000 fast-tests)
tt_blake2b.c	BLAKE2b (RFC 7693), pure C11, no SIMD, endian-safe load64/store64, self-test with known-answer vectors
tt_verify.c	Read-only store verification: explicit .ttd magic scan BEFORE reading records, CREATE size checks, CREATE_DEDUP block existence + re-hash validation, MODIFY delta decode chain, corruption counters
tt_delta.c	xdelta3 encode/decode wrapper (memory API)
tt_filter.c	Static exclusion filter: .git, node_modules, __pycache__, target, .cache, .timetravel, editor temp files (.goutputstream-*, .~lock.*, .#*, #*#)

tt_exclude.c	v1.5 per-repo exclude patterns: fnmatch + prefix + component matching, v1.6 encrypted storage (exclude.enc) with legacy fallback, pending excludes applied at repo adoption
tt_autotag.c	v1.5 semantic auto-tag generation from deltas, v1.6 garbage filtering (no-vowel tokens, high-digit ratio), binary detection (NUL bytes, non-printable ratio, high-byte ratio), stop-word list, changed line range extraction
tt_annotation.c	v1.5 annotation persistence: encrypted storage (annotations.enc) with legacy TSV fallback, per-(ts, path) lookup, atomic rename writes
tt_cache.c	v1.5 per-repo state cache: TtStateEntry with anchored/baseline_pending flags, active slot recycling, heap-allocated with doubling growth
tt_debounce.c	Event coalescing: 200 ms threshold, CLOSE_WRITE handling, per-repo heap queue (512 entries), oldest-slot recycling
tt_util.c	Pure utilities: logging, timestamp monotonic guard, file I/O (read/write_all), mkdir_p, path sanitization, external diff fork, /proc parsing (CPU%, RSS), time expression parser (relative + absolute), formatting, v1.6 path resolution with single-file support, overlap checking

2.1 THE MULTI-REPO MONOLITH

v1.3 replaces the single-repo daemon with a monolith that owns an array of repositories (up to TT_MAX_REPOS = 16). Each repository is an autonomous unit with its own watched root, its own .timetravel store, its own in-memory cache, its own debounce queue, and its own store writer.

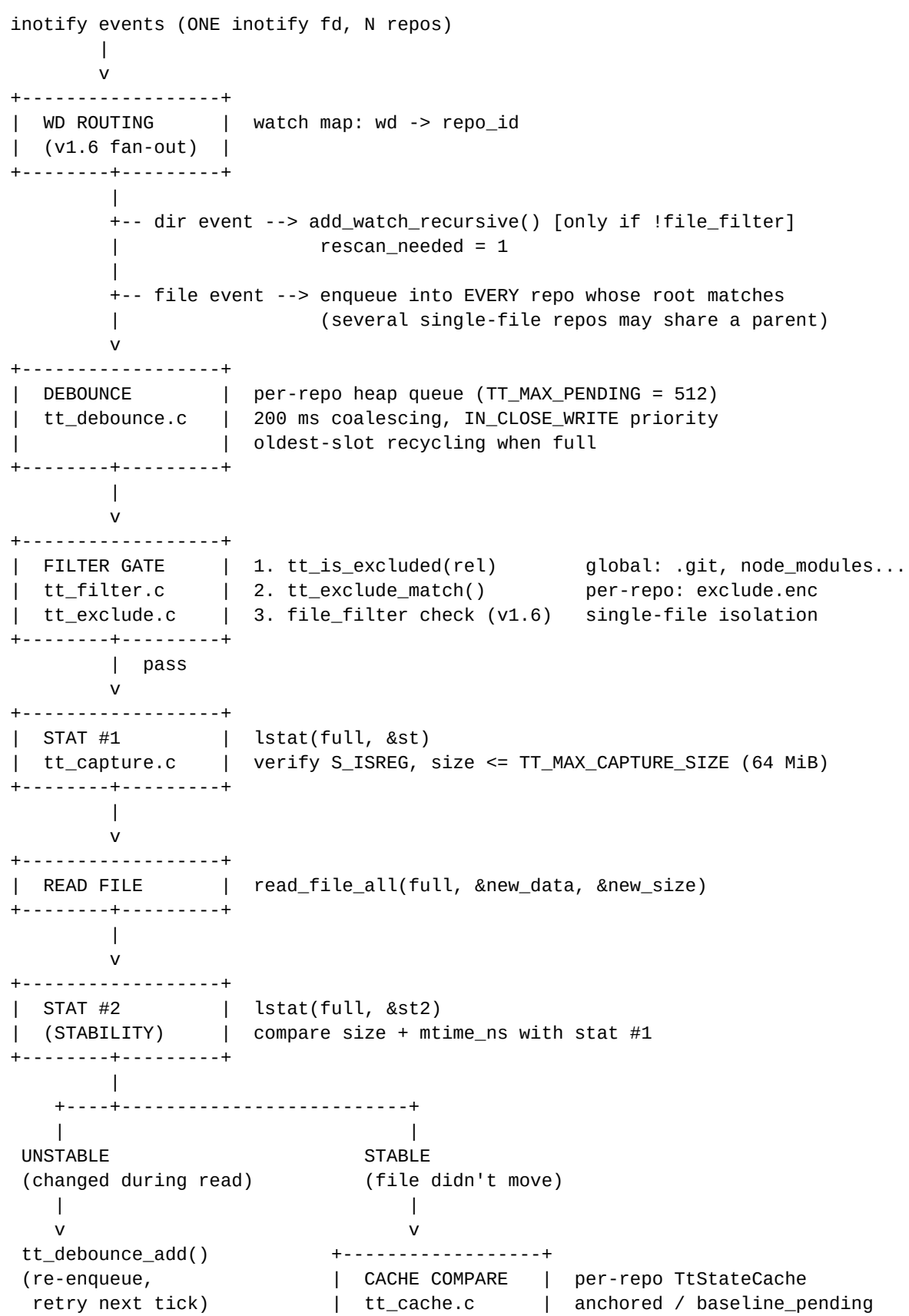
```
TtDaemon (the monolith)
|- shared inotify fd (one for ALL repos)
|- shared timer fd (tick)
|- IPC socket fd (hot add/remove)
|- watch map:  inotify wd  ->  repo id   (event routing)
|- repos[0..TT_MAX_REPOS-1]:
TtLocalRepo
|- root / store_dir (.timetravel)
|- TtStateCache    (per-repo in-memory cache)
|- TtDebounce      (per-repo coalescing queue)
|- store writer    (per-repo .ttd)
```

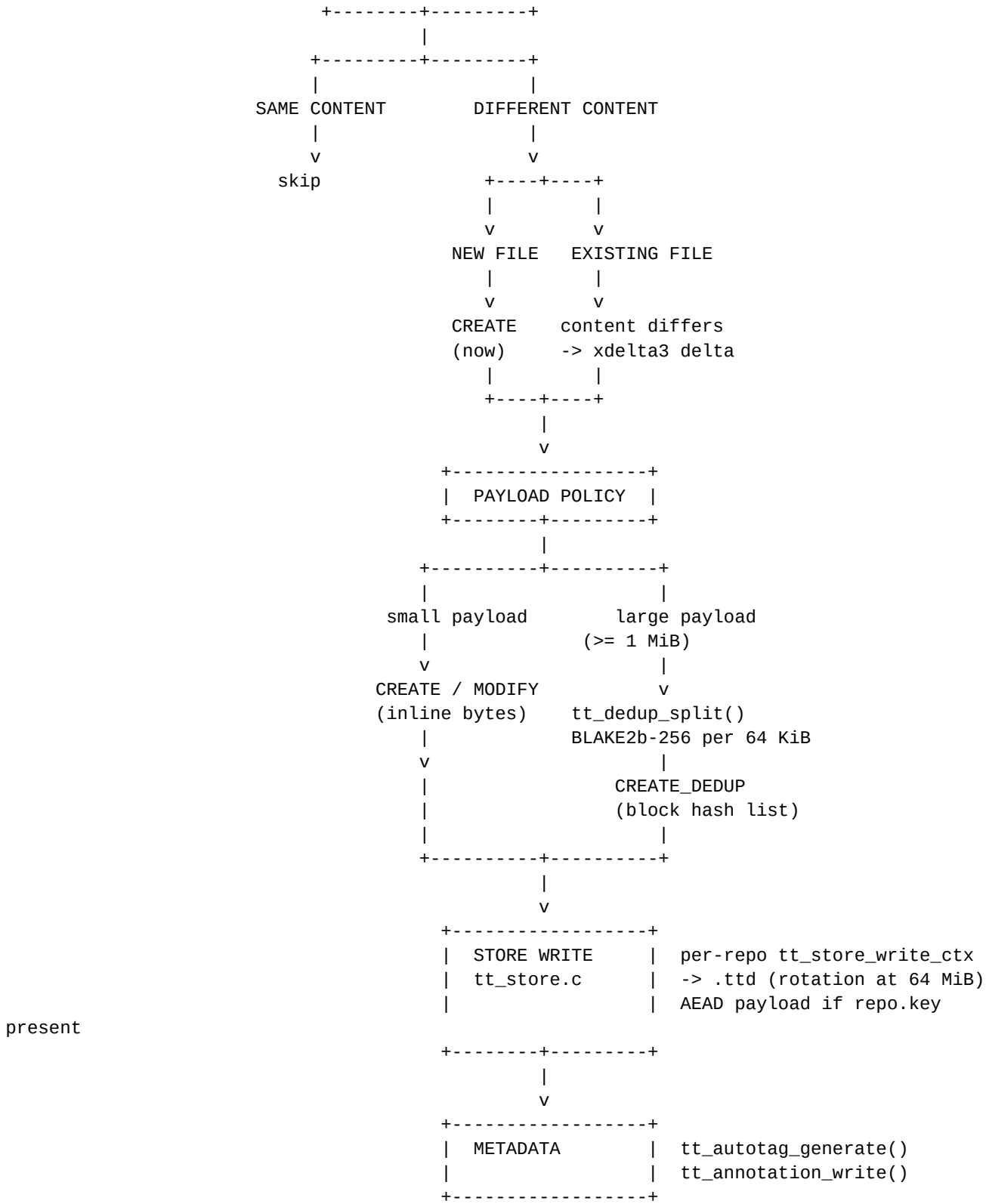
Key properties:
- A single inotify instance serves every repo. Each watch descriptor is mapped back to its repo, so an event in repo A is never recorded in B.

- Repos are added/removed at runtime over a Unix domain socket (.timetravel/timetravel.ipc). A per-user registry under /tmp/timetravel-<uid>/ provides global discovery (socket path, pid, and a global status snapshot) so any CLI instance can find the monolith.

- Concurrency safety at startup is guaranteed by a boot lock (flock) plus
IPC discovery: whichever invocation wins the lock becomes the daemon, and
every other concurrent invocation detects it and joins as a client.

2.2 CAPTURE PIPELINE DIAGRAM





PARALLEL PATHS (not event-driven):

```
  +------------------+     every TT_RESCAN_MS (10 s)
  |  PERIODIC RESCAN | --> tt_capture_rescan_and_sync_repo()
  +------------------+     also: root health, missing-file detection

  +------------------+     Unix socket, hot add/remove/list
  |  IPC DISPATCH    | --> tt_repo_adopt_base() / tt_repo_adopt_file()
  +------------------+     tt_repo_release(), boot lock (anti-stampede)
```

```
================================================================================
2.3 RESTORATION PIPELINE DIAGRAM
================================================================================

undo <path> --last | --initial | --to <t> | --tag <name>
        |
        v
+------------------+
|  TARGET RESOLVE  |  resolve_file_target_ns()
|                  |  "now", "prev", "first", "@ts", "N", "-N",
|                  |  "tag:name", relative expressions
+--------+---------+
         |
         v
+------------------+
|  LOAD RECORDS    |  scan rotated .ttd files for the path
|                  |  decrypt payload if crypto.meta present
+--------+---------+
         |
         v
+------------------+
| tt_reconstruct() |  unified state machine (v1.5+)
|                  |
|  CREATE          --> set baseline
|  CREATE_DEDUP    --> tt_dedup_reconstruct() from block pool
|  MODIFY          --> xdelta3 decode over current state
|  DELETE          --> clear state
+--------+---------+
         |
         v
+------------------+
|  ATOMIC WRITE    |  mkstemp -> write -> fsync -> chmod -> rename
|                  |  never expose a half-written file
+--------+---------+
         |
         v
+------------------+
|  v1.6 PRUNING    |  --initial also removes files added AFTER baseline
+------------------+

Note: undo/diff/dump/verify all share tt_reconstruct().
      There is exactly ONE reconstruction implementation.

================================================================================
2.4 EXPORT (DUMP) DIAGRAM
================================================================================

dump <path> --out <dir> [--with-diff] [--repo <dir>]
        |
        v
+------------------+
|  READ STORE      |  all records for the selected file/subtree
|                  |  per-repo reader inherits the crypto key
+--------+---------+
         |
         v
+------------------+
| RECONSTRUCT      |  sequential replay via tt_reconstruct()
| VERSIONS         |
```

```
|                  |  CREATE        -> inline state
|                  |  CREATE_DEDUP  -> tt_dedup_reconstruct()  <-- v1.6 fix
|                  |  MODIFY        -> xdelta3 decode
|                  |  DELETE        -> emit DELETE, reset state
+--------+---------+
         |
         v
+------------------+
|  WRITE FILES     |  <out>/files/<path>/vNNNNNN_<timestamp>
|                  |  <out>/diffs/<path>/NNNNNN_to_MMMMMM.diff
|                  |  <out>/manifest.tsv
+------------------+

manifest.tsv columns:
  <relpath>  <timestamp_ns>  <EVENT>  <size>  <extra>

Event naming in manifest:
  CREATE       -> CREATE
  CREATE_DEDUP -> CREATE_D     <-- test 43.4 asserts this
  MODIFY       -> MODIFY
  DELETE       -> DELETE

================================================================================
3. ON-DISK STORE STRUCTURE
================================================================================
Each watched repository keeps its OWN independent store:

<repo>/.timetravel/
*.ttd                Binary record files (rotation at 64 MB)
tags                 Tag definitions (name\ttimestamp)
timetravel.pid       Daemon PID + startup timestamp
timetravel.status    Daemon state snapshot
repos.list           Hot-added repos, re-adopted on restart (v1.3)
timetravel.ipc       Unix socket for hot add/remove (v1.3)
crypto.meta          v1.4: present only in encrypted repositories
blocks/              v1.4: dedup block pool
xx/
yyyyy...     BLAKE2b-named 64 KiB blocks

In addition, v1.3 keeps a PER-USER global registry:
/tmp/timetravel-<uid>/
ipc.path             Path of the monolith's IPC socket + pid
status               Global swarm snapshot (all repos)
boot.lock            flock used to serialize concurrent startups

v1.5 metadata files (all AEAD encrypted with per-repo key):
annotations.enc    Per-record semantic autotags
baseline.enc       Baseline file paths (for undo dir --initial)
exclude.enc        Exclusion patterns
repo.key           32-byte per-repo encryption key (0600)
NOTE: baseline.enc is stored PER-REPOSITORY, not globally:
<repo>/.timetravel/baseline.enc

v1.6:
+ FILE REPOS: Watch individual files instead of entire directories. Perfect for
  config files, single scripts, or isolated modules. Shares the same daemon and
  store infrastructure as directory repos.

+ EXTLOG: New command for detailed semantic history. Shows scope, revision, size,
  delta ratio, elapsed time, and a semantic summary of changes. Works with --since filter.
```

```
+ AUTO TAG CLEANLINESS: Improved token extraction and filtering. Better handling
  of binary files and garbage tokens ensures cleaner log output.

v1.5 - AUTOTAGS + METADATA ENCRYPTION:
+ SEMANTIC AUTOTAGS:
- log output now shows semantic tags for each event:
MODIFY  +1/-1 lines 3 #max_retries  worker.c
- Tags extracted from changed lines only (not the whole file).
- Token extraction excludes language keywords and common tokens.
- Maximum 4 tags per event, sorted by frequency.
- Annotations stored in .timetravel/annotations.enc (AEAD encrypted).

+ TRANSPARENT METADATA ENCRYPTION:
- annotations.tsv -> annotations.enc (encrypted)
- baseline.list -> baseline.enc (encrypted)
- exclude.list -> exclude.enc (encrypted)
- Per-repo 32-byte key auto-generated in .timetravel/repo.key
- XChaCha20-Poly1305 AEAD with random nonce per file.
- Zero user interaction: encryption is fully transparent.
- Backward compatible: legacy plaintext files are still readable.

+ MODULAR ARCHITECTURE REFACTOR:
- tt_capture.c: capture pipeline extracted from tt_main.c
- tt_repo.c: repo lifecycle management
- tt_cmds.c: CLI command handlers
- tt_reconstruct.c: unified reconstruction engine
- tt_exclude.c: exclude pattern management
- tt_autotag.c: semantic tag generation
- tt_annotation.c: annotation persistence

+ UNIFIED RECONSTRUCTION ENGINE:
- Single tt_reconstruct_file() replaces 4 duplicated implementations
- Used by: undo, diff, dump, verify
- Supports CREATE, MODIFY, DELETE, CREATE_DEDUP events
- Proper error propagation for corrupted dedup blocks

+ STATUS ENHANCEMENT:
- Global status now shows Records: line

+ TEST BATTERY EXPANDED:
- 124+ assertions across 51 sections (was 116/43)
- Section 44: exclude pattern persistence and encryption
- Sections 45-51: file repos, extlog, autotag cleanliness

v1.4: relative paths of files present at initial
capture (used by undo dir --initial for pruning)

================================================================================
4. BINARY FORMAT
================================================================================
Compatibility: TTDLTA01. Everything little-endian.
Magic:     TTDLTA01  (8 bytes)
Version:   1         (4 bytes, uint32 little-endian)

Record format (packed, little-endian):
Offset  Size    Field
------  ------  -----
0       8       timestamp_ns   (uint64)
8       1       event_type     (uint8: CREATE=1, MODIFY=2, DELETE=3)
9       4       path_len       (uint32)
13      4       delta_size     (uint32)
```

```
17      8       file_size      (uint64)
25      var     path           (path_len bytes, no NUL)
25+pl   var     payload        (delta_size bytes)

Total record size: 25 + path_len + delta_size bytes.
Event types:
TT_EV_CREATE        = 1   Full content
TT_EV_MODIFY        = 2   xdelta3 delta against previous version
TT_EV_DELETE        = 3   Deleted file (no payload)
TT_EV_RENAME        = 4   Reserved
TT_EV_CREATE_DEDUP  = 5   v1.4: content stored as dedup block hashes

Encryption (v1.4):
When repository encryption is enabled, record payloads are stored as:
nonce(24) || ciphertext(delta_size) || tag(16)

The on-disk delta_size field contains the STORED size, i.e. plaintext
size + 24-byte nonce + 16-byte tag.

The reader decrypts the payload and reports the original plaintext size.

Record headers and paths are included in the AEAD AAD. They are
authenticated but NOT encrypted. Therefore, timestamps, paths and file
sizes remain visible in encrypted repositories.

================================================================================
5. DELTA ENCODING WITH XDELTA3
================================================================================
When a file changes, the engine attempts to encode the difference as an
xdelta3 delta against the previous cached version.

Conditions to produce a MODIFY record (delta):
1. The file has a previous version in cache (anchored=1).
2. The new content differs from the cached content.
3. The delta size is strictly less than the new file size.

If the delta is greater than or equal to the new file size (common with
very small files), a full CREATE record is written instead. This is correct
behavior: the restoration logic handles both CREATE and MODIFY.

Measured savings:
A 600-byte file with a 20-byte addition produces a ~30-byte delta. Without
delta encoding, the full 620 bytes would be stored. Savings factor: ~20x.

Thresholds:
TT_MAX_CAPTURE_SIZE    64 MB    Larger files are ignored.
Delta fallback         None     Automatic: delta >= new -> CREATE.

================================================================================
6. CAPTURE MODEL (COPY-ON-FIRST-CHANGE)
================================================================================
Time-Travel DOES NOT create an initial backup/snapshot on startup. This was
a deliberate design decision to avoid writing records for files that never
change.

How it works:
1. On daemon startup (or when a repo is added), all existing files are read
into a per-repo in-memory cache (TtStateCache). Each entry is marked as
baseline_pending=1 (no history) or anchored=0 (has history from previous
sessions).
```

2. NO records are written on startup. Zero.

3. When a file is modified for the first time:
- If baseline_pending: writes CREATE [original] (backdated to daemon
startup) + MODIFY/CREATE for the new content.
- If anchored but content differs from store: writes CREATE (reanchor)
for the new content.
- If anchored and content matches: writes MODIFY (delta).

4. Subsequent changes produce MODIFY (delta) or CREATE (fallback).
Why backdate CREATE [original] to daemon startup?
So that undo --tag or undo --to with a timestamp between startup and the
first change can find the original content. Without this backdating, the
original would have a timestamp later than the tag, making it invisible
for temporal restoration.

Recovery after daemon restart:
- Files with history in store: cached with anchored=0. First change triggers
reanchor (CREATE with full content).
- Files without history: cached with baseline_pending=1. First change writes

CREATE [original] + new version.
- The per-repo start timestamp is reset to current time on every adoption.

==
7. CLI - COMMAND REFERENCE
==
General syntax:
timetravel <command> [options...]

==
7.1 DAEMON LIFECYCLE (MULTI-REPO)
==
start <dir|file>
Start the monolith with <dir|file> as repo #0.
If a monolith is ALREADY running, <dir|file> is added
to it via IPC instead of starting a second daemon.

start <path> [path2] [path3] ... (max 10)
Start the monolith with one or more paths as repos.
Each path can be a file or directory.
If a monolith is ALREADY running, paths are added via IPC.

Examples:
timetravel start src/
timetravel start file.c
timetravel start src/ file.c config.yaml
timetravel start *.c   # shell expands globs
timetravel start src/*.c   # shell expands globs

add <dir|file>
Hot-add <dir|file> to the running monolith via IPC.
If no monolith is running, bootstraps one (same
as start). Safe to call concurrently: a boot lock
guarantees only ONE daemon is ever created.

stop
Stop the whole monolith (all repos).
stop --repo <dir>   Remove ONLY <dir> from the monolith (selective).
The daemon exits only when the last repo is removed.

```
restart <dir>
Stop + start.
watch <dir> [-f]       Watch in foreground (-f) or background (default).

Concurrency guarantees (v1.3):
- Parallel `start`/`add` are serialized by a flock boot lock; exactly one
daemon is created and the rest join it as IPC clients.
- A repo that overlaps an already-watched repo (ancestor or descendant) is
rejected with an error.


================================================================================
7.2 RESTORATION
================================================================================
undo <path> [options]    Restore file or directory
--last                 Previous version (per-file for directories)
--initial              First captured version (directory state at baseline).

For directories: restores files present at the initial
capture and DELETES files added afterwards.
--to <expr>            Time expression: exact ("2026-09-13 00:07:08")
or relative ("10 minutes ago")
--tag <name>           Named checkpoint
--repo <dir>           Repository to use
--force                Skip confirmation for directory restorations

UNDO SEMANTICS:
undo dir --last        Per-file restore. Each file reverts to its own
previous version. It is not a global snapshot and
does not necessarily delete newly created files.
undo dir --initial     Restores the directory to the exact state it had
when the repo was first adopted. Files added after
that point are DELETED.
undo --to <expr>       Restores content that existed at the given time.

Does NOT delete current files that did not exist
at that time.

================================================================================
7.3 INSPECTION AND COMPARISON
================================================================================
diff <path> [options]    Unified diff
--repo <dir>           Repository to use
--to <expr>            Final revision or time point

Behavior:
- Without --from: Compares revision selected by --to against current file.
If --to is omitted, compares against previous revision.
- With --from and --to: Compares two historical revisions.
- With --from and no --to: Compares --from revision against current file.

Examples:
timetravel diff src/main.c --repo src
timetravel diff src/main.c --to prev --repo src
timetravel diff src/main.c --to 15 --repo src
timetravel diff src/main.c --to "2 hours ago" --repo src

NOTE: diff does not support comparing two historical revisions directly.

To see the delta between revision N-1 and N, use:
```

```
timetravel dump src/main.c --repo src --out /tmp/tt_main --with-diff
less /tmp/tt_main/diffs/src main.c/000014_to_000015.diff
tag <name>               Create named checkpoint at current instant
tags                     List all tags with timestamps
status                   GLOBAL swarm status (v1.3): lists every repo the
monolith watches, from ANY working directory. No

--repo needed.
status --repo <dir>      Per-repository status: daemon state, records,
bytes, uptime, CPU, RAM for that repo.
log <path> [--since exp] File change history
extlog <path> [--since exp] Extended log with semantic tags (v1.6)

================================================================================
7.4 EXPORT
================================================================================
dump <path> [options]    Export reconstructed versions
--repo <dir>           Repository to use
--out <dir>            Output directory (mandatory)
--with-diff            Also generate diffs between consecutive versions

Examples:
timetravel dump src/main.c --repo src --out /tmp/tt_main
timetravel dump src --repo src --out /tmp/tt_src --with-diff
timetravel dump . --repo src --out /tmp/tt_all --with-diff

================================================================================
7.5 MAINTENANCE
================================================================================
verify [--repo <dir>]
Verify repository integrity in read-only mode.

Checks performed:
- TTDLTA01 magic is validated on all .ttd files before reading records.
- Records are read and grouped by path.
- Per-path event chains are verified.
- TT_EV_CREATE: payload size must match file_size.
- TT_EV_CREATE_DEDUP: every referenced dedup block must exist, have the
expected size, and its BLAKE2b-256 hash must match the reference.
- TT_EV_MODIFY: the xdelta3 delta must decode against the reconstructed
previous state.
- TT_EV_DELETE: clears the reconstruction state for that path.
- Encrypted repositories: AEAD authentication is enforced.

Returns 0 when the store is clean.
Returns non-zero when corruption, tampering, missing dedup blocks, invalid
magic, or authentication failure is detected.
compact [--repo <dir>]

Collapse delta chains and rewrite the store atomically.

In v1.4, compact also runs dedup GC:
orphan blocks not referenced by any record are removed.

For encrypted repositories, compact requires the correct passphrase.
If the repository cannot be read/authenticated, compact aborts and
GC does not delete blocks.

start <dir> --encrypt
Create a new encrypted repository.
Prompts for passphrase twice and creates crypto.meta.
```

```
================================================================================
7.6 TIME AND REVISION EXPRESSIONS
================================================================================
Exact timestamps:
"2026-09-13 00:07:08"  (Format: YYYY-MM-DD HH:MM:SS)

Relative times:
"10 seconds ago"
"5 minutes ago"
"2 hours ago"
"1 day ago"
"1 week ago"
"now"

================================================================================
7.7 CONFIRMATION BEHAVIOR
================================================================================
Directory undo without --force asks for confirmation when stdin is a
terminal (isatty). When stdin is not a terminal (pipe, /dev/null, script),
fgets returns immediately: NULL = cancel, "y"/"Y" = continue.

This prevents hangs in automated scripts while preserving interactive safety.

================================================================================
8. REVISION AND TIME EXPRESSIONS
================================================================================
In commands like diff and undo, expressions can indicate revisions, tags,
timestamps, or relative instants.

Revision expressions:
now / head / latest      Last saved revision
prev / previous          Previous revision
first / initial          First saved revision
N                        Revision number N (1 = first)
-N                       N revisions before the last
0                        Empty / non-existent file
tag:<name>               Existing tag
@<timestamp>             Internal timestamp (seconds or nanoseconds)

Time expressions:
Exact: "2026-09-13 00:07:08"
Relative: "10 sec", "5 min", "2 hours", "1 day", "1 week", "now"

Important:
- `diff --to N` compares revision N against the current file.
- `diff` does not support comparing two historical revisions.

Use `dump --with-diff` to see inter-revision deltas.
- `undo --to "2026-09-13 00:07:08"` restores the state exactly at that time.
- `undo --to "10 minutes ago"` restores the state 10 minutes prior to now.
- `undo dir --last` restores each file to its own previous version but
does NOT delete files created after the session started.
- `undo dir --initial` restores to baseline and DELETES files added later.
- `dump --with-diff` automatically generates all consecutive diffs.

================================================================================
9. VERSION AND DIFF EXPORT (DUMP)
================================================================================
The dump command reconstructs full versions from CREATE/MODIFY/DELETE
records and writes them to an output directory.
```

Useful for:
- Auditing file changes.
- Reviewing 10, 100, or more consecutive diffs.
- Visually comparing source code evolution.
- Debugging reconstructions.
- Extracting old content without doing an undo.

Typical output structure:
<out>/
manifest.tsv
files/
<sanitized_name>/
v000001_<timestamp>
v000002_<timestamp>
v000003_<timestamp>
diffs/
<sanitized_name>/
000001_to_000002.diff
000002_to_000003.diff
000003_to_000004.diff

The manifest.tsv file contains one line per event/version, with fields:
path   timestamp   event   size   version_file

Practical examples:

Export a file:
timetravel dump src/brs_tui.c --repo src --out /tmp/tt_tui

Export with diffs:
timetravel dump src/brs_tui.c --repo src --out /tmp/tt_tui --with-diff

View the last 100 diffs:
find /tmp/tt_tui/diffs -name '*.diff' | sort | tail -n 100 | less

View diff between revision 14 and 15:
less /tmp/tt_tui/diffs/brs_tui.c/000014_to_000015.diff

Count exported versions:
tail -n +2 /tmp/tt_tui/manifest.tsv | wc -l

==========
10. CODE QUALITY AND COVERAGE (170 ASSERTIONS / 52 SECTIONS)
==========
Time-Travel v1.6 includes a 52-section regression battery.

The battery is split into:
- 170 functional assertions (sections 1-40 + sections 42-52).
- 8 Valgrind memory-audit assertions (section 41).

When the battery is executed against a ThreadSanitizer build, the Valgrind
memory-audit section is automatically skipped. In that mode the summary
normally shows:

Passed: 169/169
Failed: 0
Skipped: 1

This is expected: TSan and Valgrind must not be combined.

```
Run the normal battery:
./compile.sh test
./mega_test.sh ./build/timetravel

Run the ASan/UBSan battery:
./compile_asan.sh
./mega_test.sh ./build_san/timetravel

Run the TSan battery:
./compile_tsan.sh
TSAN_OPTIONS="halt_on_error=1:second_deadlock_stack=1" \

./mega_test.sh ./build_tsan/timetravel

Battery structure:
Sections 01-09:   Core functionality
Sections 10-12:   Dynamic behavior
Section  13:      Git clone
Sections 14-15:   Edge cases
Section  16:      Compaction
Section  17:      Directory undo
Sections 18-20:   Stress
Section  21:      Special characters
Sections 22-23:   Daemon lifecycle
Section  24:      Tags
Section  25:      Diff
Section  26:      Directory undo confirmation
Section  27:      Log filter
Sections 28-35:   Multi-repo monolith
Section  36:      Store verification
Section  37:      Dedup large file capture + restore
Section  38:      Dedup GC orphan cleanup
Section  39:      XChaCha20-Poly1305 encryption
Section  40:      Encryption hardening
Section  41:      Valgrind memory audit
Section  42:      undo dir --initial baseline prune
Section  43:      dump dedup large file regression (CREATE_D + MODIFY)
Section  44:      exclude pattern persistence and encryption
Sections 45-48:   File repos (v1.6)
Section  49:      Extlog (v1.6)
Section  50:      Autotag cleanliness (v1.6)
Section  51:      Fix verification (small file MODIFY)
Section  52:      Multiple files at start


The battery is self-cleaning: it kills any leftover daemons and clears the
global registry on startup and on exit, so orphan processes from previous
runs (or manual testing) cannot pollute a run.

================================================================================
10.1 CERTIFICATION SUMMARY
================================================================================
+--------------------------------+----------+--------------------------------+
| Test                           | Result   | What it demonstrates           |
+--------------------------------+----------+--------------------------------+
| Full battery (52 sections)     | 176/176  | Functional + skip Valgrind     |
| Functional battery             | 176/176  | Core, dedup, crypto, verify,   |
|                                |          | dump regression, file repos    |
| Valgrind memory audit          | 168/170  | Crypto skipped (KDF too slow)  |
| TSan battery                   | 175/176  | Valgrind skipped under TSan    |
| QEMU aarch64 battery           | 175/176  | Portable to non-native arch    |
```

ASan+UBSan build	PASS	Memory errors / UB detection
ThreadSanitizer	PASS	No data-race reports
Valgrind memory audit	PASS	0 leaks, 0 errors
QEMU aarch64 portability	PASS	Runs under qemu-user emulation
Store verify clean	OK	Healthy store returns OK
Store verify corrupt	OK	Corruption detected
Invalid .ttd magic detection	OK	verify fails before reading
Dedup large restore	OK	2 MiB restored from blocks
Dedup GC	OK	Orphan removed, refs kept
Encrypted store	OK	No plaintext in .ttd
Encrypted undo	OK	Correct passphrase restores
Wrong passphrase	OK	File remains unchanged
Encrypted compact	OK	No plaintext after compact
Encrypted GC fail-closed	OK	Wrong key does not delete
Corrupted dedup block	OK	Restore fails gracefully
Tampered encrypted .ttd	OK	AEAD authentication fails
Corrupted crypto.meta	OK	Unlock rejected
File repo isolation	OK	Sibling files not captured
Extlog semantic tags	OK	Tags extracted correctly
Multiple files at start	OK	start accepts N files
Glob expansion (*.c)	OK	Shell glob resolves correctly
>10 files rejection	OK	Hard limit enforced

10.2 FORENSIC MEMORY AND CONCURRENCY AUDITS

Time-Travel v1.6 can be audited with several independent toolchains.

AddressSanitizer + UndefinedBehaviorSanitizer (ASan/UBSan)

Build:

```
./compile_asan.sh
```

Run:

```
./mega_test.sh ./build_san/timetravel
```

ASan detects:

- heap buffer overflows;
- use-after-free;
- memory leaks (when leak detection is enabled);
- invalid stack/global memory access.

UBSan detects:

- signed integer overflow;
- null pointer dereference;
- misaligned access;
- other undefined behavior.

Do NOT run ASan builds under Valgrind.

ThreadSanitizer (TSan)

Build:

```
./compile_tsan.sh
```

Run strict mode:

```
TSAN_OPTIONS="halt_on_error=1:second_deadlock_stack=1" \
```

```
./mega_test.sh ./build_tsan/timetravel

Run audit/log mode:
TSAN_OPTIONS="halt_on_error=0:exitcode=0:second_deadlock_stack=1:log_path=$PWD/build_tsan/
tsan.log" \
./mega_test.sh ./build_tsan/timetravel

Then inspect logs:
ls -lh build_tsan/tsan.log.*
grep -a "WARNING: ThreadSanitizer" build_tsan/tsan.log.*

Important TSan notes:
- TSan detects data races and some deadlock patterns.
- TSan cannot be combined with ASan.
- TSan cannot be combined with Valgrind.
- mega_test automatically skips the Valgrind memory audit when TSan is active.
- TSan log files are usually created only when there is something to report.

If no build_tsan/tsan.log.* files exist and the strict run passes, no
races were detected during that execution.

--------------------------------------------------------------------------
Valgrind Memcheck
--------------------------------------------------------------------------
Valgrind is used for leak checking and memory-error auditing.
The dedicated section 41 exercises:
- start
- stop
- verify
- undo
- compact
- status
- log

Expected result:
- definitely lost: 0 bytes
- indirectly lost: 0 bytes
- possibly lost: 0 bytes
- ERROR SUMMARY: 0 errors

Do NOT run Valgrind on ASan or TSan builds.
Use a native or Valgrind-compatible binary for Valgrind audits.

==========================================================================
10.3 VALGRIND / SANITIZER POLICY (V1.4)
==========================================================================
WARNING: Due to hardware and time constraints, Valgrind instrumentation is
DISABLED by default for encrypted test sections 39 and 40 when running the
test battery under Valgrind.

Reason:
- PBKDF2-HMAC-BLAKE2b with 100000 iterations is extremely slow under
Valgrind (10-20x slower than native execution).
- XChaCha20-Poly1305 AEAD and dedup block encryption multiply the cost.
- Target development hardware (Core 2 Duo / mechanical HDD) cannot
complete these sections in a reasonable time under Valgrind.

What IS tested under Valgrind:
- Sections 1-38: core functionality, dedup, GC, compaction.
- Section 41: dedicated Valgrind memory audit (leaks, errors).
```

What is NOT tested under Valgrind by default:
- Section 40: Encryption extended hardening (functional).
These sections ARE fully tested in the normal (non-Valgrind) battery run.

Additional sanitizer policy:
- TSan builds automatically skip section 41.
- ASan/UBSan builds must not be executed inside Valgrind.
- TSan builds must not be executed inside Valgrind.
- ASan and TSan must not be combined in the same build.

If you want to enable encrypted sections under Valgrind:
1. Set SKIP_CRYPTO_UNDER_VALGRIND=0 in mega_test.sh
2. Increase crypto timeouts: CRYPTO_TIMEOUT=600

WARNING: Enabling these sections may increase total test time from
~15 minutes to several hours on the target hardware.

==
11. DESIGN DECISIONS
==
Baseline manifest (v1.4):
undo dir --initial restores the directory to its exact state at the moment
the repo was first adopted. A baseline.list file records which files existed
at that point. Files added afterwards are deleted during --initial restore.

The manifest is written once on first adoption and never overwritten,
preserving the original capture point across daemon restarts.

Multi-repo monolith (v1.3):
A single daemon owns up to 16 repositories. This avoids a process per
folder, keeps one shared inotify instance and one timer, and lets the user
attach/detach folders on the fly. Each repo is still fully autonomous: its
own .timetravel store, cache, and debounce queue. The monolith is an
orchestration layer, not a coupling layer.

Boot lock + IPC discovery (v1.3):
Concurrent `start`/`add` must never create two daemons. The winner of a
flock boot lock becomes the daemon and publishes its IPC socket; every other
invocation discovers it and joins as a client. The lock acquisition is
bounded (~30 s) so a client can never hang forever, and the daemon releases
the lock as soon as it is discoverable.

Hot add/remove over a Unix socket (v1.3):
Repositories are attached and detached at runtime through
.timetravel/timetravel.ipc. A per-user registry under /tmp/timetravel-<uid>/
stores the socket path, pid, and a global status snapshot so any CLI
instance can locate the monolith regardless of the current directory.

Signal handlers live only in the daemon (v1.3):
Client processes (start/add/stop) keep default signal dispositions so that
external timeouts (e.g. `timeout` in the test battery) can terminate a stuck
client with SIGTERM. Only the long-lived daemon installs a handler.

No initial snapshot:
Files are cached in memory on startup but NOT written to disk. Records are
only created on the first change (copy-on-first-change). Original content
is backdated to daemon startup so undo --tag/--to works for pre-existing
files. This avoids writing thousands of records on startup for repos that
might never change.

Atomic restorations:

All writes use mkstemp + write + fsync + chmod + rename. The daemon never
sees a half-written file during restoration. Corruption window: zero, even
under power loss.

Stability verification:
Double-stat (size + mtime in nanoseconds) before recording prevents
capturing files during concurrent writes. Unstable reads are re-enqueued
via debounce for retry on the next tick.

Per-file directory undo:
undo dir --last restores each file to its own previous version, not to a
single global timestamp. This matches the user expectation of "undo my last
changes" rather than "revert to a specific moment".

No fsync per record:
Records are visible via page cache instantly. Durability is bounded by file
rotation and close. Massive performance gain in bursts (500 files in <2
seconds vs minutes with fsync per record).

Manual-only compaction (v1.3):
The automatic 03:00 compaction was removed to keep the daemon's behavior
deterministic and to avoid surprising rewrites during long sessions. Delta
chains are still collapsed on demand with `timetravel compact`.

Robust exclusions:
Directory exclusions (.git, node_modules, etc.) use component matching at
any depth via path_has_component(), not fnmatch with FNM_PATHNAME which
fails on deep absolute paths. File patterns are compared only against the
base name.

Daemon lifecycle via PID file + global registry:
PID + startup timestamp stored in .timetravel/timetravel.pid, plus a
per-user global registry for cross-directory discovery. A second start in
the same repo detects the active daemon and exits cleanly. Stop sends
SIGTERM, waits 5 seconds, escalates to SIGKILL.

Session resumption:
When restarting the daemon over the same repository, it attempts to continue
the last valid .ttd. This avoids unnecessarily fragmenting history into
multiple small files. If the last .ttd is corrupt, locked, or exceeds 64 MB,
a new one is created.

Export vs undo:
undo restores files in the working tree.
dump exports reconstructed versions outside the working tree.
dump is a read-only operation on history and modifies neither the watched
repo nor the store.

12. LIMITS AND CONSTANTS

TT_PATH_MAX	4096	Maximum path length
TT_MAX_REPOS	16	Max repositories per monolith (v1.3)
TT_DEBOUNCE_MS	200	Event coalescing window
TT_TICK_MS	50	Timer tick interval
TT_RESCAN_MS	10000	Periodic rescan interval (10 s)
TT_COMPACT_THRESHOLD	15	Records per chain before collapse
TT_MAX_PENDING	4096	Maximum pending debounce entries
TT_MAX_CAPTURE_SIZE	64 MB	Larger files are ignored
TT_STORE_MAX_FILE_SIZE	64 MB	.ttd rotation threshold
TT_DEDUP_BLOCK_SIZE	64 KiB	Dedup block size

TT_DEDUP_MIN_SIZE	1 MiB	Minimum file size for dedup capture
TT_DEDUP_HASH_LEN	32	BLAKE2b-256 hash length
TT_KEY_LEN	32	Symmetric key length
TT_NONCE_LEN	24	XChaCha20 nonce length
TT_TAG_LEN	16	Poly1305 tag length
TT_KDF_ITERATIONS	100000	PBKDF2-HMAC-BLAKE2b iterations (release)
TT_KDF_ITERATIONS_TEST	1000	PBKDF2-HMAC-BLAKE2b iterations (test)

NOTE: TT_COMPACT_HOUR (automatic 03:00 compaction) was REMOVED in v1.3.
Compaction is now manual only via `timetravel compact`.

13. POSSIBLE ISSUES

Daemon won't start:
Verify musl-gcc is installed: which musl-gcc
On Debian/Ubuntu: sudo apt install musl-tools
Check that third_party/xdelta/xdelta3/xdelta3.c exists.

`add` says the repo overlaps another:
You are trying to watch a folder that is inside an already-watched
folder, or that contains one. This is rejected by design to avoid
double-capturing the same files. Remove the conflicting repo first
(stop --repo <dir>) or choose a non-overlapping path.

`status` shows no repos:
Either no monolith is running, or a leftover global registry from a
previous run is stale. Run `timetravel start <dir>` to launch one. If
you suspect orphan daemons, kill them and clear /tmp/timetravel-<uid>/.

undo doesn't restore the expected version:
For files that existed before daemon startup, original content is only
recorded on the FIRST change. If no change has occurred since startup,
there is no record to restore from.
Solution: make any edit to the file, then undo will work.

undo dir hangs waiting for confirmation:
This happens when stdin is a terminal and --force is not used.
Press 'y' + Enter to continue, or 'n' + Enter / Ctrl+C to cancel.

In scripts, use a pipe:
echo "y" | timetravel undo dir --last ...
Or use --force to skip confirmation entirely.
diff shows "(no differences)" unexpectedly:

There are three normal causes:
1. The file only has one record (first capture).
2. The selected version is identical to the current file.
3. You are using --to N expecting to see the change introduced by N.
Case 3 is the most confusing.

Example:
timetravel diff src/main.c --repo src --to 15
This compares revision 15 against the current file, NOT revision 14
against revision 15.

To see the change introduced by revision 15, use dump:
timetravel dump src/main.c --repo src --out /tmp/tt_main --with-diff
less /tmp/tt_main/diffs/src main.c/000014_to_000015.diff

Large files aren't captured:

Files exceeding TT_MAX_CAPTURE_SIZE (64 MB) are silently ignored. A
warning is logged in the daemon log. This limit exists to prevent
excessive memory usage during capture and restoration.

.git files appear in history:
This shouldn't happen. The exclusion filter uses component matching at
any depth. If it does, verify tt_filter.c is compiled and linked (check
that compile.sh includes src/tt_filter.c).

Store grows indefinitely:
Compaction is MANUAL in v1.3:
timetravel compact --repo /path/to/project
Compaction collapses delta chains longer than TT_COMPACT_THRESHOLD.

I want to see many diffs, not just one:
Use dump:
timetravel dump <path> --repo <repo> --out /tmp/tt_dump --with-diff
Then:
find /tmp/tt_dump/diffs -name '*.diff' | sort | less
or:
find /tmp/tt_dump/diffs -name '*.diff' | sort | tail -n 100 | less

Encrypted repository won't unlock:
Verify that you are using the correct passphrase and that
.timetravel/crypto.meta exists and is not corrupted.
Without crypto.meta, the repository cannot be unlocked.
I forgot the passphrase:
The passphrase cannot be recovered.
The encryption key is derived from it and is not stored.

compact asks for passphrase:
This is expected for encrypted repositories.
If the passphrase is wrong, compact should fail and dedup GC
should not delete blocks.

verify fails:
verify returns non-zero when the store is corrupted or when
encrypted records fail authentication.
This can happen after manual modification of .ttd files,
disk corruption, or using the wrong passphrase.

blocks/ directory grows:
The blocks directory stores deduplicated content.
Blocks referenced by any historical record are preserved.
Orphan blocks are removed by compact/GC.

Large files take longer to capture:
Files >= 1 MiB are split into 64 KiB blocks, hashed with BLAKE2b
and stored in the dedup pool. This is expected.

Valgrind is killed when running a TSan binary:
This is expected. ThreadSanitizer and Valgrind cannot be combined.
mega_test detects TSan builds and skips the Valgrind memory audit section.

If you need Valgrind results, run it against a native or Valgrind-compatible
binary, not against build_tsan/timetravel.
TSan does not create log files:
ThreadSanitizer usually creates log_path.<pid> files only when it has
something to report.

If you used:

TSAN_OPTIONS="...:log_path=/path/to/tsan.log"
and there are no files like:
tsan.log.12345
then TSan probably did not detect any warning during that run.
To confirm the binary is instrumented:
nm build_tsan/timetravel | grep -m 5 __tsan
You should see symbols such as:
U __tsan_init
U __tsan_read1
U __tsan_write1
ASan binary fails under Valgrind:

Do not combine AddressSanitizer with Valgrind.
Use the ASan build for ASan/UBSan testing and a native/Valgrind build for
Valgrind memory auditing.

mega_test shows Passed: 116+/116+ and Skipped: 1 under TSan:

This is expected. The 116+ functional assertions are evaluated, and the
Valgrind memory audit section is skipped because TSan is active.

==========
14. DEDUP BLOCK STORAGE (V1.4)
==========
When a captured file is at least TT_DEDUP_MIN_SIZE (1 MiB), Time-Travel
splits its content into TT_DEDUP_BLOCK_SIZE blocks (64 KiB).

Each block is hashed with BLAKE2b-256 and stored in the dedup pool:
.timetravel/blocks/<first 2 hex chars>/<remaining hex chars>
The .ttd record for a deduplicated file stores only the ordered list of
32-byte block hashes, not the file content itself.

Properties:
- Identical blocks are stored only once.
- Identical blocks can be shared across files and versions.
- Restore reconstructs the file by reading blocks from the pool.
- Orphan blocks can be removed by compact/GC.
- In encrypted repositories, blocks are encrypted individually.
- dump correctly reconstructs and exports deduplicated files
(TT_EV_CREATE_DEDUP events are supported).

Dedup is especially effective for:
- large files with partially repeated content;
- VM images, disk images, archives;
- large generated files with stable regions.

Dedup is less effective for:
- fully random data;
- already compressed data;
- encrypted data with high entropy.

==========
15. ENCRYPTION (V1.4)
==========
Time-Travel v1.4 supports optional per-repository encryption using
XChaCha20-Poly1305 (AEAD).

Creating an encrypted repository:
timetravel start /path/to/project --encrypt
The command prompts for a passphrase twice and creates:
.timetravel/crypto.meta

```
crypto.meta contains:
- magic identifier;
- random 16-byte salt;
- KDF iteration count;
- key verifier.
The passphrase itself is NEVER stored.

Key derivation:
PBKDF2-HMAC-BLAKE2b(passphrase, salt, iterations) -> 32-byte key
Default iteration count:
TT_KDF_ITERATIONS = 100000 (release)
TT_KDF_ITERATIONS_TEST = 1000 (test)

Encrypted data:
- .ttd record payloads;
- dedup blocks.
Authenticated but not encrypted:
- record headers;
- file paths;
- timestamps;
- stored size metadata.
Therefore, encryption protects content but does not hide file names,
timestamps or repository structure.

Important:
- If the passphrase is lost, the repository cannot be recovered.
- crypto.meta must be preserved. Without it, unlocking is not possible.
- Wrong passphrase is detected via the key verifier.
- Tampered encrypted records fail AEAD authentication.
- Tampered crypto.meta is rejected.

================================================================================
15.1 SEMANTIC AUTOTAGS (V1.5)
================================================================================
Time-Travel v1.5 automatically generates semantic tags for every capture
event. These autotags appear in the `log` and `extlog` command output:

MODIFY    +1/-1 lines 3 #max_retries                          worker.c
CREATE    CREATE (176 B) #futex_lock #PTHREAD_MUTEX_INITIALIZER  worker.c

Format:
<EVENT>  <quantity> <range> <tags>  <path>
Quantity:
+A/-R lines    Lines added/removed (or replaced)
EDIT [Lx]      Content modified without changing the line count
Range:
N              Single affected line
N-M            Range of affected lines
Tags:
#identifier    Relevant identifiers extracted ONLY from the changed
lines (maximum 4, sorted by frequency).
Language keywords and common tokens are excluded.

Storage:
Autotags are stored in .timetravel/annotations.enc (AEAD encrypted).
Internal format: timestamp_ns \t path \t summary

Files:
tt_autotag.c       Tag generation (tokenization, ranking)
tt_autotag.h       Public API
```

```
tt_annotation.c    Encrypted annotation persistence
tt_annotation.h    Public API

================================================================================
15.2 METADATA ENCRYPTION (V1.5)
================================================================================
v1.5 automatically encrypts repository metadata files.
The encryption is TRANSPARENT: the user does not need to do anything.

Encrypted files:
annotations.enc    Per-record autotags (formerly: annotations.tsv)
baseline.enc       Baseline file paths (formerly: baseline.list)
exclude.enc        Exclusion patterns (formerly: exclude.list)

Repository key:
Each repo has a 32-byte key automatically generated in
.timetravel/repo.key (0600 permissions).
This key is used to encrypt/decrypt metadata.
The user NEVER needs to interact with this key.

Encrypted file format:
"TTENC01" || nonce(24) || tag(16) || ciphertext

Algorithm:
XChaCha20-Poly1305 AEAD (same primitive as payload encryption).

Backward compatibility:
If legacy plaintext files exist (annotations.tsv, baseline.list,
exclude.list), they are read directly. New files are always written
encrypted.

Security:
- An attacker with disk access sees only random bytes.
- The repo.key has 0600 permissions (owner only).
- If repo.key is lost, metadata cannot be recovered.
- File content remains protected by payload encryption (v1.4) or
plain deltas.

Functions:
tt_repo_get_key()           Generates/reads the repo key
tt_crypto_encrypt_file()    Encrypts a complete file
tt_crypto_decrypt_file()    Decrypts a complete file

================================================================================
16. FAQ - FREQUENTLY ASKED QUESTIONS
================================================================================

================================================================================
16.1 CAN I USE TIME-TRAVEL ON WINDOWS OR MAC?
================================================================================
NO. Time-Travel uses Linux kernel-specific interfaces:
- inotify for filesystem events
- timerfd for periodic ticks
- rename() atomicity guarantees (POSIX)
- mkstemp for secure temporary files

There is no portability layer. It is Linux-only by design.

================================================================================
16.2 DOES IT WORK OVER NFS / SSHFS / NETWORK MOUNTS?
================================================================================
```

Partially. inotify DOES NOT work over NFS or SSHFS. The daemon will start
but won't receive events. The periodic rescan (every 10 s) will still
capture changes, but with higher latency.

For network directories, consider running the daemon on the server and
accessing the store remotely, or use a tool designed for network backup
(BareSnap, Restic, Borg).

==
16.3 HOW MUCH DISK SPACE DOES THE STORE NEED?
==
General rule: the store needs between 0.5x and 1x the size of the unique
data being tracked, depending on change frequency.

Factors:
- Delta encoding: small changes produce small deltas (~20-30 bytes for a
line edit in a 600-byte file).
- Full copies: very small files (<~50 bytes) are always stored as full
CREATE because deltas are larger.
- Compaction: collapses long delta chains into base + super-delta.
- Binary files: photos, videos, compressed archives don't generate good
deltas; each version stores almost the full size.

To check current usage:
timetravel status --repo /path/to/project

==
16.4 CAN I BACKUP THE .TIMETRAVEL DIRECTORY?
==
YES, but it's not recommended. The .timetravel directory contains all
history. Backing it up creates a circular dependency.

Better approach: backup the SOURCE files with a traditional tool (BareSnap,
rsync, tar). The .timetravel store is a local undo buffer, not a backup.

==
16.5 WHAT HAPPENS IF THE DAEMON CRASHES?
==
The store is NEVER left in a corrupt state:
- Records are written atomically (all or nothing).
- Without fsync per record, some recent records might be lost on power
loss, but existing records remain intact.
- On restart, the daemon rebuilds its cache from disk and continues.
- The PID file is cleaned on graceful exit; obsolete PIDs are detected via
kill(pid, 0) and overwritten on new startup.

==
16.6 CAN TWO DAEMONS WATCH THE SAME DIRECTORY?
==
NO. In v1.3 there is only ever ONE daemon (the monolith). A second `start`
or `add` detects the running monolith via the boot lock + IPC discovery and
joins it instead of spawning a duplicate. Concurrent startups are
serialized so exactly one daemon is created. Overlapping repos are rejected.

Use `timetravel stop --repo <dir>` to detach a repo, or plain `stop` to halt
the whole monolith.

==
16.7 CAN I WATCH SEVERAL UNRELATED FOLDERS AT ONCE? (NEW IN V1.3)
==
YES. This is the headline feature of v1.3. Start the monolith with one

folder and attach the rest hot:
timetravel start ~/src
timetravel add ~/docs
timetravel add /mnt/data/notes

Each folder keeps its own isolated .timetravel store. Inspect the whole
swarm at any time with plain `timetravel status`.

==
16.8 IS THERE ENCRYPTION?
==
YES, starting with v1.4.
Time-Travel supports optional repository encryption using
XChaCha20-Poly1305.

Create an encrypted repository with:
timetravel start /path/to/project --encrypt

Encryption protects:
- record payloads;
- dedup blocks.

Encryption does NOT hide:
- file paths;
- timestamps;
- file sizes;
- record metadata.
Those fields are authenticated but remain visible.

IMPORTANT:
- The passphrase is not stored anywhere.
- If you lose the passphrase, the repository cannot be recovered.
- crypto.meta must be preserved.
- Back up crypto.meta together with the repository.

==
16.9 CAN I SYNC THE STORE TO ANOTHER MACHINE?
==
Not natively. The store is a local undo buffer. For remote backup, use a
dedicated tool (BareSnap with SSH, rsync, rclone).

You CAN rsync the .timetravel directory as a rudimentary sync mechanism, but
concurrent access from two machines is unsupported and will corrupt the store.

==
16.10 HOW DO I RESUME A SESSION?
==
Simply restart the daemon over the same repository:
timetravel stop --repo /path/to/project
timetravel start /path/to/project

If the last .ttd is valid and hasn't exceeded the limit, the daemon will
continue writing to it. Hot-added repos are re-adopted automatically from
repos.list. To check session state:
timetravel status --repo /path/to/project

==
16.11 CAN I SEE 100 DIFFS?
==
Yes.
The recommended way is:

```
timetravel dump <path> --repo <repo> --out /tmp/tt_dump --with-diff

Then:
find /tmp/tt_dump/diffs -name '*.diff' | sort | tail -n 100 | less
The practical limit isn't 100; it depends on available history, file sizes,
and disk space for export.

================================================================================
16.12 HOW DO I SEE THE CHANGE INTRODUCED BY A SPECIFIC REVISION?
================================================================================
diff compares a historical revision against the current file, not against
another historical revision. To see the delta introduced by revision N,

use dump instead:
timetravel dump src/main.c --repo src --out /tmp/tt --with-diff
less /tmp/tt/diffs/src main.c/000014_to_000015.diff
This generates all consecutive diffs, so you can inspect any inter-revision
change.

================================================================================
16.13 CAN I USE EXACT DATES FROM THE LOG?
================================================================================
YES. `undo --to` accepts exact timestamps in the format
"YYYY-MM-DD HH:MM:SS" as shown in the log output.

Example:
timetravel log src/main.c --repo src
# Output shows: 2026-09-13 00:06:17  CREATE  ...  src/main.c
timetravel undo src/main.c --to "2026-09-13 00:06:17" --repo src

This restores the file to the exact state recorded at that second.

================================================================================
17. COMPARISON WITH OTHER TOOLS
================================================================================
+--------------------+-------------+----------+----------+----------+
| Feature            | Time-Travel | git      | Restic   | BareSnap |
+--------------------+-------------+----------+----------+----------+
| Purpose            | Real-time   | Version  | Backup   | Backup   |
|                    | file undo   | control  |          |          |
| Language           | Pure C11    | C/Perl   | Go       | Pure C11 |
| Static binary      | ~292 KB     | ~40 MB   | ~40 MB   | ~2.1 MB  |
| Runtime deps       | NONE        | perl     | none     | none     |
| Real-time capture  | YES (inotify)| NO      | NO       | NO       |
| Multi-repo daemon  | YES (v1.3)  | NO       | NO       | NO       |
| Hot add/remove     | YES (v1.3)  | NO       | NO       | NO       |
| File repos         | YES (v1.6)  | NO       | NO       | NO       |
| Delta encoding     | xdelta3     | binary   | NO       | xdelta3  |
| Deduplication      | YES         | YES      | YES      | YES      |
| Encryption         | XChaCha20   | NO*      | AES-256  | XChaCha20|
| Remote support     | NO          | YES      | S3/SFTP  | Native SSH|
| Per-file undo      | YES         | checkout | extract  | extract  |
| Directory undo     | YES         | checkout | restore  | restore  |
| Named checkpoints  | YES (tags)  | YES      | YES      | YES      |
| Diff               | YES         | YES      | YES      | YES      |
| Inter-revision diff| YES         | YES      | NO       | NO       |
| Export versions    | YES (dump)  | export   | extract  | extract  |
| Manual compaction  | YES         | gc       | prune    | prune    |
| Initial backup     | NO          | clone    | init     | init     |
+--------------------+-------------+----------+----------+----------+
* git supports GPG signing but not repository encryption.
```

Time-Travel occupies a unique niche: real-time per-file undo for active
development, now across many folders from a single daemon. It is NOT a
backup tool. Use it alongside a proper backup solution (BareSnap, Restic,
Borg) for disaster recovery.

18. RESUMING A SESSION

Time-Travel automatically resumes the session when you restart the daemon
over the same repository.

To stop and resume:
./build/timetravel stop --repo /path/to/project
./build/timetravel start /path/to/project

Behavior:
- Looks for the last .ttd file inside .timetravel/
- If valid, not corrupt, not locked, and under 64 MB, continues writing to it.
- If missing, corrupt, locked, or over the limit, creates a new .ttd.
- Re-adopts any hot-added repos listed in repos.list (v1.3).

Check session state:
./build/timetravel status --repo /path/to/project

View session files:
ls -l /path/to/project/.timetravel/*.ttd

Important:
There is no specific "session continue" command. The session resumes when
you run "start" again over the same repository.

19. ANECDOTE

Let me tell you an anecdote about Baresnap and Timetravel. I was in "flow mode"
in an open chat without logging in or anything-terating with an AI. After a few hours,
the browser crashed, and i thought, "Oh no... I just lost the changelog, and the AI knew
it."

I reopened the browser, started a new chat, and told the AI what had happened. It replied,
"Hey, if you had Timetravel enabled, you just need to dump the diffs; send them to me, and
i'll generate the changelog." I sent the diffs using the `dump --diff` command, the AI
analyzed them and created the changelog, and it worked!

20. CHANGELOG

v1.6 - FILE REPOS + EXTLOG + OPTIMIZED TESTS:
+ FILE REPOS:
- Watch individual files instead of entire directories.
- Perfect for config files, single scripts, or isolated modules.
- Shares the same daemon and store infrastructure as directory repos.
- Tests added for file repo isolation, overlap protection, and persistence.

+ EXTLOG:
- New command for detailed semantic history.
- Shows scope, revision, size, delta ratio, elapsed time, and semantic summary.
- Works with --since filter.
- Additive to classic log (does not replace it).

```
+ AUTO TAG CLEANLINESS:
- Improved token extraction and filtering.
- Better handling of binary files and garbage tokens.
+ FIX VERIFICATION:
- Added test for small file MODIFY fix.

v1.5 - AUTOTAGS + METADATA ENCRYPTION:
+ SEMANTIC AUTOTAGS:
- log output now shows semantic tags for each event:
MODIFY  +1/-1 lines 3 #max_retries  worker.c
- Tags extracted from changed lines only (not the whole file).
- Token extraction excludes language keywords and common tokens.
- Maximum 4 tags per event, sorted by frequency.
- Annotations stored in .timetravel/annotations.enc (AEAD encrypted).

+ TRANSPARENT METADATA ENCRYPTION:
- annotations.tsv -> annotations.enc (encrypted)
- baseline.list -> baseline.enc (encrypted)
- exclude.list -> exclude.enc (encrypted)
- Per-repo 32-byte key auto-generated in .timetravel/repo.key (0600).
- XChaCha20-Poly1305 AEAD with random nonce per file.
- Zero user interaction: encryption is fully transparent.
- Backward compatible: legacy plaintext files are still readable.

+ MODULAR ARCHITECTURE REFACTOR:
- tt_capture.c: capture pipeline extracted from tt_main.c
- tt_repo.c: repo lifecycle management
- tt_cmds.c: CLI command handlers
- tt_reconstruct.c: unified reconstruction engine
- tt_exclude.c: exclude pattern management
- tt_autotag.c: semantic tag generation
- tt_annotation.c: annotation persistence

+ UNIFIED RECONSTRUCTION ENGINE:
- Single tt_reconstruct_file() replaces 4 duplicated implementations
- Used by: undo, diff, dump, verify
- Supports CREATE, MODIFY, DELETE, CREATE_DEDUP events
- Proper error propagation for corrupted dedup blocks

+ STATUS ENHANCEMENT:
- Global status now shows Records: line

+ TEST BATTERY EXPANDED:
- 124+ assertions across 51 sections (was 116/43)
- Section 44: exclude pattern persistence and encryption
- Sections 45-51: file repos, extlog, autotag cleanliness

+ COMPILATION SCRIPTS UPDATED:
- compile.sh, compile_asan.sh, compile_tsan.sh, compile_valgrind.sh
- All include new source files

v1.4 - DEDUP + CRYPTO CORE:
+ v1.4 manual corrections:

+ dump supports TT_EV_CREATE_DEDUP:
Large deduplicated files are correctly reconstructed and exported.
Regression test added (section 43).

+ CLI semantics clarified:
- undo dir --last does not delete newly created files by default.
- undo dir --initial deletes files not present at baseline.
```

- diff does not support --from.

+ Baseline manifest (v1.4):
On first repo adoption the daemon writes .timetravel/baseline.list
with the relative paths of all files present at that moment.
undo dir --initial uses this manifest to restore the directory to
its exact initial state: files in the baseline are restored, files
not in the baseline are deleted.

+ Test battery expanded to 116+ assertions across 43+ sections.
Section 43 validates dump reconstruction of CREATE_DEDUP files.
Section 42 validates baseline prune behaviour.

+ TSan support:
- compile_tsan.sh builds a ThreadSanitizer-instrumented binary.
- mega_test skips the Valgrind memory audit automatically when TSan is active.
- TSan and Valgrind are explicitly treated as incompatible.

+ Store verification hardened:
- verify validates TTDLTA01 magic on all .ttd files before reading records.
- verify detects invalid-magic files even when the store reader skips them.
- verify checks CREATE, CREATE_DEDUP, MODIFY and DELETE chains.
- verify validates dedup block existence, size and BLAKE2b-256 hash.
- verify returns non-zero on corruption or tampering.

+ Documentation updated for v1.4 modules:
- tt_dedup.c
- tt_crypto.c
- tt_kdf.c
- tt_blake2b.c
- tt_verify.c

NOTE: Encryption protects file content and dedup blocks, but it does not
provide full metadata privacy. Paths, timestamps and file sizes remain
visible because they are required for history indexing and restoration.

+ DEDUP BLOCK STORAGE:
Files >= TT_DEDUP_MIN_SIZE (1 MiB) are captured as 64 KiB
BLAKE2b-256 content-addressed blocks.

+ Dedup pool:
Blocks are stored under:
.timetravel/blocks/<2hex>/<remaining hex>

+ TT_EV_CREATE_DEDUP event:
Deduplicated files store only the ordered list of block hashes
in the .ttd record.

+ Dedup restore:
undo reconstructs large files by reassembling blocks from the pool.

+ Dedup GC:
compact now removes orphan blocks not referenced by any record.
Non-hex garbage inside blocks/ is also removed.

+ Encrypted repository support:
New command:
timetravel start <dir> --encrypt

+ XChaCha20-Poly1305 AEAD:
Record payloads and dedup blocks are encrypted and authenticated.

+ PBKDF2-HMAC-BLAKE2b:
Passphrase-based key derivation using random salt and 100000
iterations by default.

+ crypto.meta:
Stores magic, salt, KDF iteration count and key verifier.
The passphrase itself is never stored.

+ Store verification:
New command:
timetravel verify --repo <dir>
Detects corrupted or tampered stores. For encrypted repositories,
AEAD authentication is enforced.

+ Atomic compaction:
compact writes the new store into .compact_tmp and only replaces
the live store if the rewrite succeeds.

+ Encrypted compaction:
compact preserves encryption and leaves no plaintext behind when
operating on encrypted repositories.

+ Fail-closed GC:
If compact cannot read/authenticate repository history, GC does not
delete dedup blocks.

+ Hardened restore:
Encrypted dedup restore fails gracefully when blocks are missing,
corrupted, or cannot be authenticated.

+ New test sections:
- Section 36: store verification
- Section 37: dedup large file capture + restore
- Section 38: dedup GC orphan cleanup
- Section 39: XChaCha20-Poly1305 encryption
- Section 40: encryption hardening

Compatibility notes:
- Unencrypted TTDLTA01 stores remain readable.
- Encrypted repositories require the correct passphrase and a valid
crypto.meta file.
- New TT_EV_CREATE_DEDUP records require a v1.4-aware reader.
- Encryption authenticates headers and paths via AAD but does not
encrypt them. Paths, timestamps and sizes remain visible.

v1.3 - MULTI-REPO HOT CORE:
+ MULTI-REPO MONOLITH: a single daemon now watches up to TT_MAX_REPOS (16)
repositories concurrently. Each repo keeps its own isolated .timetravel
store, cache, and debounce queue.
+ New `add` command: hot-attach a repository to the running monolith over
IPC. If no monolith is running, `add` bootstraps one (same as `start`).
+ `start` is now monolith-aware: if a monolith is already running, the new
repo is attached to it instead of refusing or duplicating.
+ Selective `stop --repo <dir>`: detach a single repo; the daemon exits only
when the last repo is removed. Plain `stop` halts the whole monolith.
+ IPC hot core (tt_ipc.c): repositories are added/removed at runtime over a
Unix domain socket (.timetravel/timetravel.ipc).
+ Boot lock: concurrent `start`/`add` are serialized via flock with bounded
acquisition (~30 s). Parallel invocations bootstrap exactly ONE daemon -
no duplicates, no hangs.

+ Global discovery & status: a per-user registry (/tmp/timetravel-<uid>/)
exposes the socket path, pid, and a swarm snapshot, so plain `status`
(no --repo) reports the whole monolith from any working directory.
+ Overlap protection: adding a repo that contains or is contained by an
already-watched repo is rejected.
+ Independent root health: deleting one repo's directory marks only that
repo as root_lost; the others keep capturing. It recovers on recreation.
+ Per-repo isolation: a corrupted or deleted store in one repo never affects
the others.
+ Persistence: hot-added repos are recorded in repos.list and re-adopted
automatically on restart.
+ Signal handlers now live only in the daemon, so external timeouts can
terminate a stuck client (SIGTERM).
- REMOVED automatic daily compaction (03:00). Compaction is manual only via
`timetravel compact` (TT_COMPACT_HOUR removed).
+ Regression battery expanded to 77/77 across 35 sections. Sections 28-35
cover the multi-repo monolith (hot add, overlap, root loss/recovery,
persistence, bootstrap, stampede, global status, selective stop).
+ Test battery is now self-cleaning (kills leftover daemons and clears the
global registry on startup and exit) to prevent cross-run pollution.

v1.2:
+ `undo --to` now accepts exact log dates in "YYYY-MM-DD HH:MM:SS" format
(e.g. "2026-09-13 00:06:17").
+ New test 5.3 validates exact timestamp restoration, eliminating the race
condition with the daemon debounce.
+ Regression battery expanded to 51/51 passing (2m 00s on Core 2 Duo).
+ Documentation updated with exact date usage examples.
+ New FAQ covering exact timestamp usage.

v1.1:
+ New `dump` command to export reconstructed versions.
+ `dump --with-diff` option to generate consecutive diffs.
+ New `diff --from` to compare two historical revisions.
+ `diff --to` expanded with revisions: N, -N, first, prev, head.
+ Support for comparing against empty file with `--to 0`.
+ Enhanced status with CPU, RAM, written deltas, and pending events.
+ Session resumption: daemon resumes last valid .ttd on restart.
+ Updated `undo --to` to accept exact timestamps and relative times.
+ Expanded CLI examples across all commands (not just diff).

v1.0:
+ Initial version
+ Real-time capture via inotify with debounce (200 ms coalescing)
+ Copy-on-first-change model (no initial backup)
+ Backdated CREATE [original] for pre-existing files
+ Per-file directory undo (--last, --initial)
+ Atomic restorations via mkstemp + rename
+ Double-stat stability verification (no ghost CREATE 0)
+ No fsync per record (visibility via page cache)
+ Robust exclusions via path_has_component at any depth
+ Daemon lifecycle: start/stop/restart with PID file
+ Named checkpoints: tag/tags/undo --tag
+ Unified diff between versions
+ Log with --since filter
+ Enhanced status (daemon state, uptime, records, bytes)
+ Developed and validated on Intel Core 2 Duo (2007 hardware)

==
21. TIMETRAVEL LICENSE
==

- SOURCE CODE (src/): Under the GNU General Public Licence v3.0 (GPLv3).
- MANUAL and ASSETS: Licensed under the GNU Free Documentation Licence (GFDL v1.3).

==========
END OF DOCUMENT
==========